genai_cz.py

GenAI a Python

Pavel Tišnovský, tisnik@centrum.cz

Obsah

- Teoretická část
  - Strojové učení a umělá inteligence
  - Modely, trénink, vyhodnocování
  - Neuronové sítě
  - Generativní umělá inteligence
  - Frameworky pro GenAI
  - Základní vlastnosti LLM
  - OpenAI a další standardizovaná API
  - Framework Llama Stack
  - Langchain pro tvorbu aplikací využívajících GenAI
  - RAG (Retrieval-augmented generation)
  - Evaluace
- Praktická část
  - Volání LLM
  - Langchain
  - Tokenizace textu
  - Tvorba embedded modelů
  - Vyhledávání založené na podobnosti vektorů
  - FAISS
  - SentenceTransformer
  - PgVector
  - Závěrečný projekt

Teoretická část

Úvod

- Umělá inteligence
- Vývoj umělé inteligence
- Strojové učení
- Vztah strojového učení a umělé inteligence

Umělá inteligence

- Definice
  - konstrukce strojů, které dokážou provádět činnosti vyžadující inteligenci, pokud by byly prováděny lidmi (Marvin Minsky, 1967)
  - existují ovšem i alternativní definice
- Modelování lidské mysli
  - shora dolů (psychologie)
  - zdola nahoru (neurověda)
  - zprostředka (informatika)
- Inteligentní chování může vzniknout ze spojení velkého množství jednoduchých systémů
  - koncept neuronových sítí

Vývoj umělé inteligence

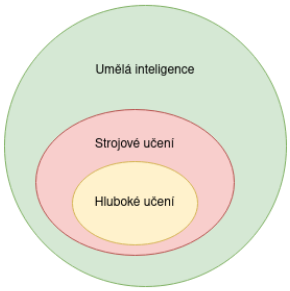
- 1943-1955
  - první myšlenky, že něco podobného může reálně vzniknout
  - booleovský model neuronu
  - A. Turing
    - Computing Machinery and Intelligence
- 1956

    - McCarthy (LISP)
    - (pravděpodobně) poprvé použil termín AI
    - Newel a Simon: Logic theorist
- velké očekávání pokroku v dalších letech
    - dařilo se částečné řešení různých problémů
    - prakticky každý měsíc nový objev
- cca 1965
    - vystřízlivění
    - existovala sice spousta vyřešených problémů, ale ty byly triviální
    - nalezeny limity, které se nedařilo překonat
    - první "AI zima"
- sedmdesátá léta
    - systémy založené na znalostech
    - vývoj v mnoha oblastech (hledání ropy atd.)
- začátek osmdesátých let
    - velké investice do AI
    - očekávání se nenaplnila
    - potom nastává druhá "AI zima"
- druhá polovina osmdesátých let
    - rozvoj neuronových sítí (což nebyla novinka)
- 1995
    - systémy SOAR (State, Operator and Result)
- 2000
    - big data (v tom pokračujeme i dnes)
    - ale prozatím žádné větší objemy
    - "wow" efekt na úrovni AI
- 2010
    - deep learning
- 2020
    - LLM (prompt engineering, ne fine tuning)
- současnost
    - LLM
    - generativní AI

Strojové učení

- podoblast umělé inteligence
- změna interního stavu systému při tréninku
- několik způsobů realizace strojového učení
- "statistické učení"
- typicky se nejedná o plně automatizovanou činnost
    - vyžaduje chytrá (strategická) rozhodnutí
    - výběr modelu
    - výběr hyperparametrů modelu
    - rozdělení vstupních dat
    - filtrace dat
- nalézají se skryté vzorky a vazby v datech

Vztah strojového učení a umělé inteligence



- Umělá inteligence (AI)
    - strojové učení (machine learning, ML)
    - hluboké učení (deep learning, DL)
    - robotika
    - neuronové sítě (NN)
    - zpracování přirozeného jazyka (NLP)
- AI > ML > NN > DL
- Umělá inteligence
    - objevování
    - odvozování

    - odůvodnění
- Strojové učení
    - (sofistikovaná) analýza
    - predikce (!)
    - rozhodování (klasifikace nebo regrese)

Proč strojové učení?

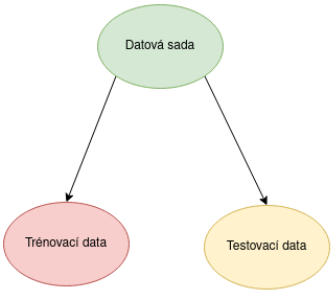
- Chceme, aby se stroj naučil řešit zadaný problém na základě vzorových řešení:
    - řešení je příliš komplikované
    - problém se často mění, vyvíjí
    - lidská práce je drahá (v porovnání se strojovou)
    - máme k dispozici tolik dat, že je není možné zpracovat "ručně"

Typické aplikace strojového učení

- Rozpoznávání vzorů
    - věci/osoby/výrazy tváře na fotkách
    - mluvená slova
    - spam
    - medicínská diagnóza
- Rozpoznávání anomálií
    - netypické sekvence finančních transakcí
    - netypická data přicházející ze senzorů v atomové elektrárně
- Předpovídání
    - vývoj ceny akcií na burze / vývoj měnového kurzu
    - jaké filmy bude mít daný člověk rád
    - věk osoby na fotografii
- Shlukování
    - vyhledávání zpráv s podobným obsahem
    - vyhledání skupin zákazníků s podobnými vlastnostmi

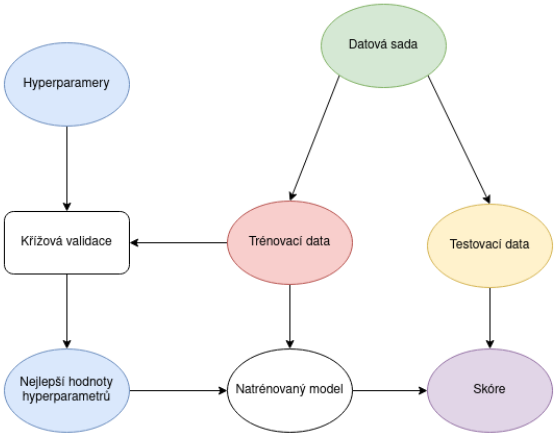
Základní pojmy

- Datová sada
    - trénovací data
    - testovací data



Techniky strojového učení

- Supervised learning
    - také se nazývá "predictive modeling"
    - známe takzvané "kategorie" neboli odpovědi
- Unsupervised learning
    - neznáme odpovědi
    - model musí najít struktury/vzory v datech
    - typicky různé varianty clusteringu



Supervised learning

1. trénink na základě vstupních dat
   - model se naučí vztahy mezi daty a očekávanou odpovědí
2. predikce na základě jiných(!) dat
   - problematika rozdělení dat
3. výsledky
   - klasifikace: koupí si A, B nebo C?
   - regrese: vektor příznaků, numerická hodnota nebo hodnoty

---

Unsupervised learning

1. trénink modelu na základě vstupních dat
   - ovšem bez znalosti správných odpovědí
2. shluková analýza
3. latentní a faktorová analýza

Další možnosti

1. kombinace obou metod (bez/s učitelem)
2. učení se zpětnou vazbou
   - pasivní
   - aktivní

Modely strojového učení

- ANN
- Desicion trees
- Support-vector machine
- Regresní analýza
- Bayesovské sítě
- Genetické algoritmy
- Neuronové sítě

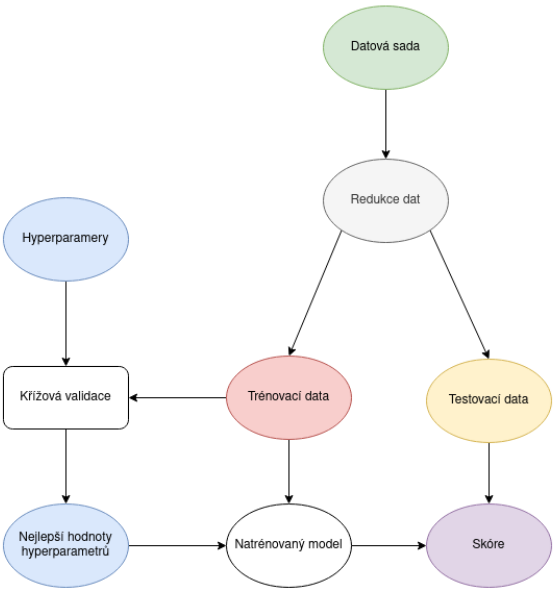
Jak začít?

1. jaké atributy použít z dat?
2. jaký model vybrat
3. jak optimalizovat pro větší výkon
4. jak vytvořit model, který bude vhodný pro pro něj neznámá data?
5. jak odhadnout vhodnost modelu pro neznámá data?

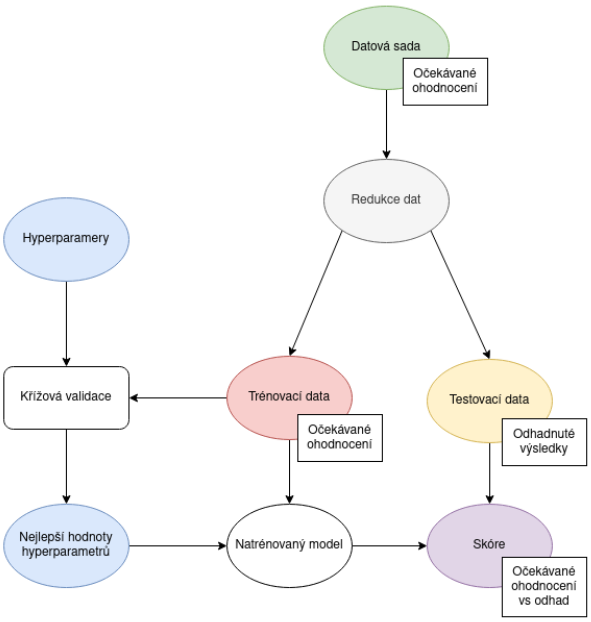
Komprimace dat

- souvislost mezi ML a komprimací dat
- predikce
- tzv. optimální komprese
  - při predikci lze použít aritmetické kódování

Redukce dat



Redukce dat

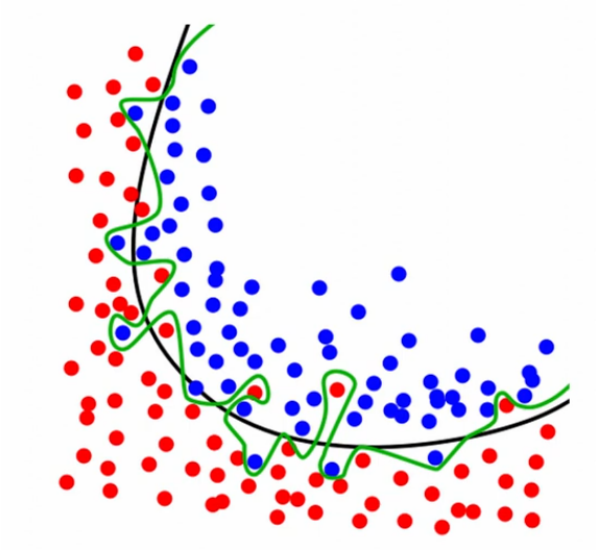


Nedoučení a přeučení

- Nedoučení
  - malá sada dat, na kterých je model trénován
  - příliš složitý model
  - data reprezentují pouze malý vzorek celého spektra hodnot
- Přeučení
  - velká vazba na trénovací data
  - menší flexibilita práce s daty, která model nezná
  - použití polynomu vyššího stupně, když by stačila lineární regrese

---

Přeučení



Úspěšnost modelu

- Pro zcela nová (neznámá) data!
  - ne pro trénovací množinu
  - častá chyba

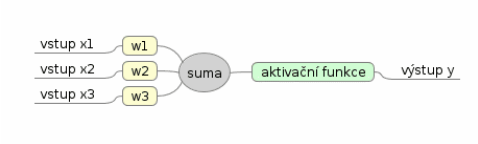
Křížová validace (cross validation)

- rozdělení dat do bloků
  - například na 1/10
  - 9/10 pro trénink
  - 1/10 pro otestování

Neuronové sítě

- Propojení takzvaných neuronů
  - Model neuronu
  - Způsob propojení neuronů
  - Vstupy a výstupy

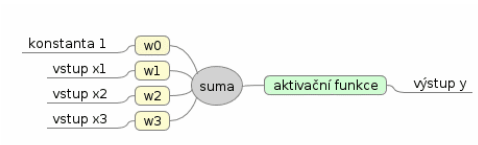
Model neuronu



- libovolný počet vstupů
- typicky jeden výstup
- váhy vstupů
- aktivační funkce

y = f(w1x1 + w2x2 + … + wnxn)

Bias

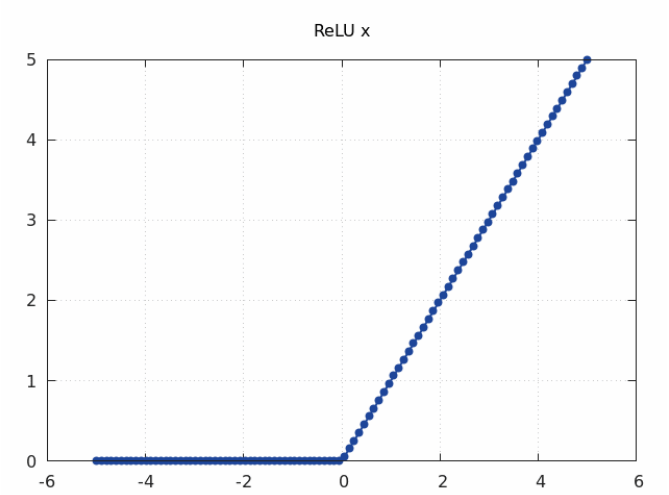


y = f(w0 + w1x1 + w2x2 + … + wnxn)

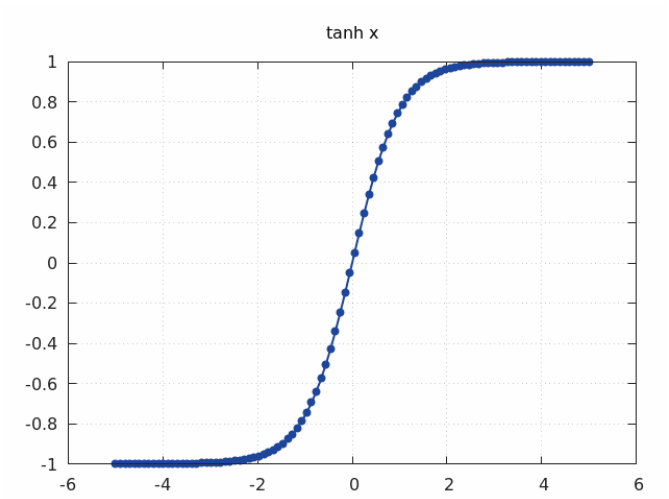
Aktivační funkce

- Jediná nelinearita v modelu
- Mnoho typů aktivačních funkcí

Aktivační funkce



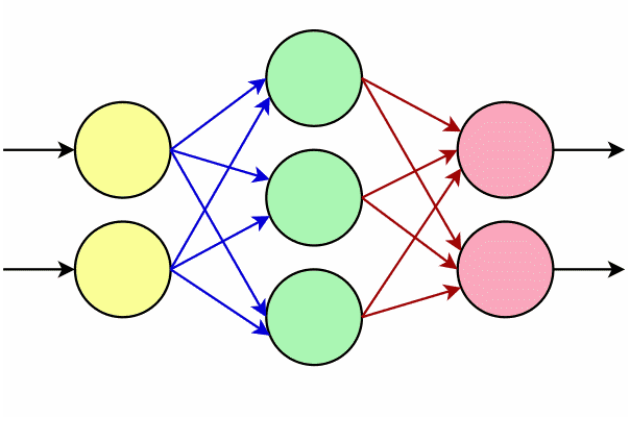
Aktivační funkce



Feed-forward síť

- Vstupní vrstva
- Skryté vrstvy
- Výstupní vrstva

Feed-forward síť



Příliš mnoho vrstev

- Model se přestane učit nebo se učí velmi pomalu
  - vanishing gradient problem
  - "méně je někdy více"

Konvoluční neuronové sítě

- Typicky pro rastrové obrázky
  - mírné posunutí, zkosení atd.
  - lze sice řešit klasickými NN
  - ovšem je to zbytečně složité (RAM, CPU čas)

  - příliš mnoho stejných, ale samostatně uložených vah
  - konvoluční a subsamplingové vrstvy

Konvoluční neuronové sítě

- Typická konfigurace
  - vstupní vrstva
  - konvoluční vrstva #1
  - subsamplingová vrstva #1

Generativní AI

Frameworky pro GenAI

- langchain
- langgraph
- autogen
- metaGPT
- phidata
- CrewAI
- pydanticAI
- controlflow
- langflow
- LiteLLM (???)
- Llama Stack

Potřebujeme sofistikované frameworky?

- některé základní operace je možné provést i jen s využitím "pouhých" knihoven
- příkladem může být knihovna OpenAI
- a do určité míry taktéž LiteLLM

Výpis všech dostupných modelů

- používá se přímo balíček openai
- klíč se získává z proměnné prostředí

```python
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
models = client.models.list()

for model in models:
    print(model.id)
```

Potřebujeme sofistikované frameworky?

- Kvóty
- Konverzace
- Kontext
- Guardrails
- RAG
- BYOK
- MCP
- Agenti
- A2A

Llama Stack

Co je to Llama Stack?

- Framework pro tvorbu aplikací s AI
  - chat boti
  - obecně generativní AI (GenAI)
  - trénink, evaluace
- Skutečný framework nezávislý na programovacím jazyku
  - systém providerů
  - RAG, správa kvóty, guardrails, metriky atd.

Nejjednodušší využití Llama Stacku

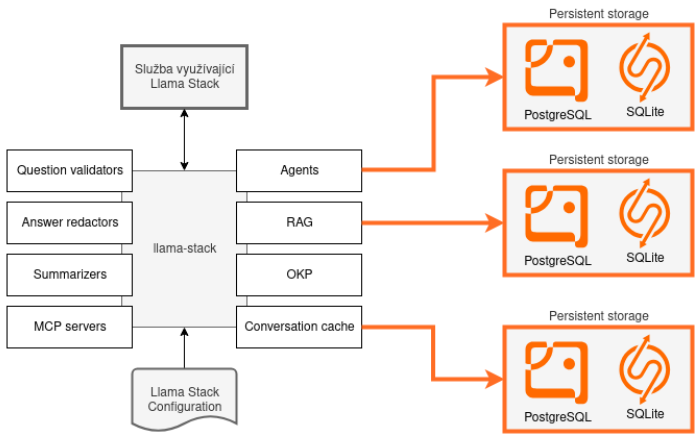
- Volání LLM
- Zpracování odpovědi od LLM
- "chatbot v.0.0.1"

Skutečné požadavky jsou však širší

- Podpora RAG
- Historie konverzace
- Rozvětvení konverzace
- Zapamatování fakt o uživateli
- Volání MCP
- Kvóty
- Kontrola dotazů a odpovědí
- Volání více LLM, evaluace odpovědí atd.

API a poskytovatelé

- Plně konfigurovatelné
- Lze získat seznam dostupných API
- Taktéž lze získat seznam dostupných poskytovatelů
- Pozor: závislosti vyžadované jednotlivými poskytovateli



Poskytovatelé

- Agents
  - interakce se systémem agentů (samotní agenti mohou provádět různé operace)
- Inference
  - rozhraní k LLM modelům i k embedding modelům
- VectorIO
  - původně rozhraní kvektorovým databázím (hledání podobných vektorů)
  - nyní i fulltext hledání
- Safety
  - detekce dotazů s nevhodným či nepovoleným obsahem

apod.
- Telemetry
  - telemetrie
  - OpenTelemetry ale i další
- Eval
  - vyhodnocování odpovědí modelů atd.
- DatasetIO
  - čtení a zápisy datových sad z/do zvoleného systému
  - může být i lokální souborový systém

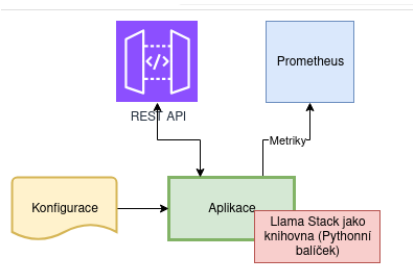
Komunikace s Llama Stackem

- CLI
- REST API
- Jako běžná knihovna (Python atd.)
- Llama Stack klient
  - podporuje REST API
  - podporuje i běh formou knihovny (async)

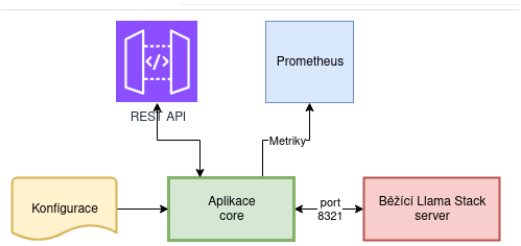
Llama Stack klient

- Python
- Swift
- Kotlin
- Node.js

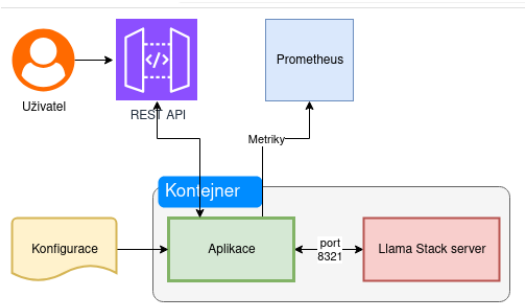
Llama Stack jako knihovna



Llama Stack jako samostatná služba

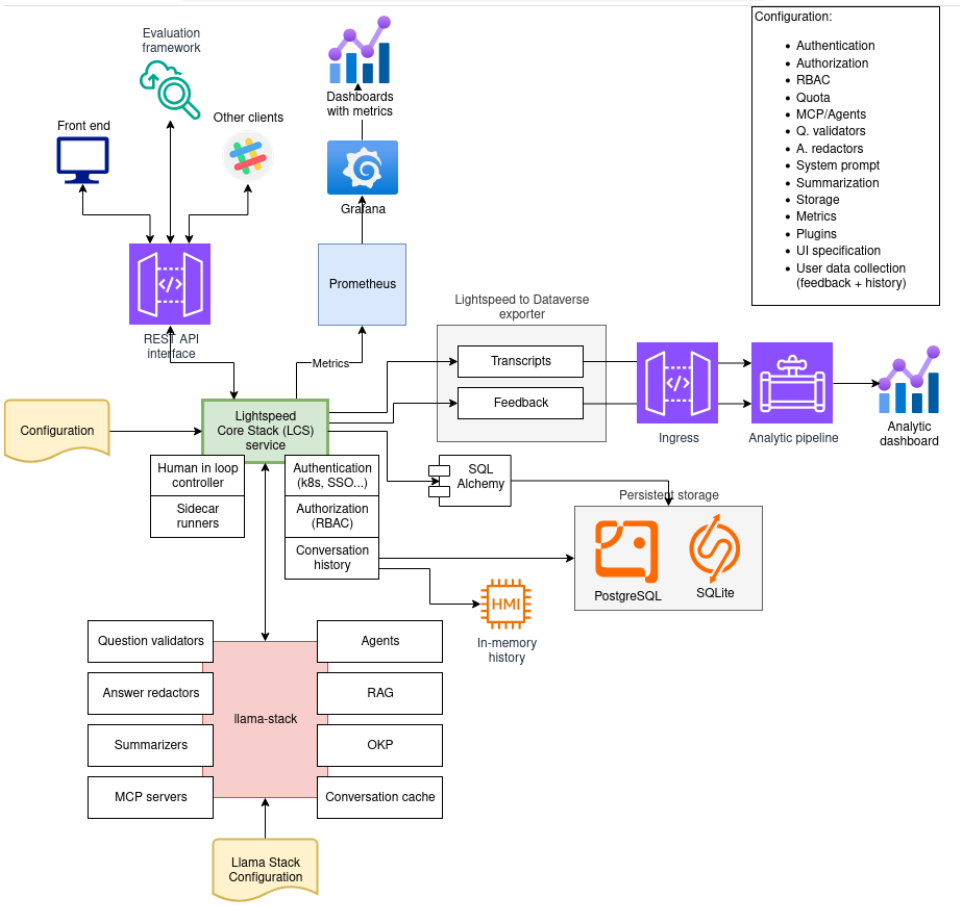


Běh v kontejneru



Příklad služby postavené na Llama Stacku

- REST API postavené nad API Llama Stacku
- Obě možnosti spuštění Llama Stacku
- Implementace formou asynchronního kódu (Python)



LangChain

Langchain

- Framework pro GenAI a Python
- Chatboti
- RAG
- Paměť konverzací
- Shrnutí konverzací
- Generování syntetických dat

Langchain

- Jednoduché úkoly je možné řešit jednoduše
- Mnohdy pouze několikařádkové skripty
- Operátor | pro tvorbu "kolon"
- Napojení na RAG

Evaluace

Proč provádět evaluaci?

- Celková výkonnost
- Uživatelská zkušenost
- Detekce zkreslení odpovědí
- Dodržování etických a právních předpisů

Přednosti

- Zlepšení odpovědí (detekce halucinací atd.)
- Ladění modelu řízené daty
- Benchmarky: porovnání modelů
- Zlepšení spolehlivosti v průběhu času

Lightspeed evaluation framework

- Podpora více LLM
- Ragas, DeepEval + vlastní implementace
- Evaluace celé konverzace
- Podpora nástrojů (tools) a agentů
- Závěrečná statistická analýza

Praktická část

- Volání LLM
- Langchain
- Tokenizace textu
- Tvorba embedded modelů
- Vyhledávání založené na podobnosti vektorů
- PgVector
- FAISS
- Evaluace

Llama Stack klient

- Využijeme klienta pro Python

```
uv init
uv add llama-stack-client
```

Llama Stack běží jako samostatná služba

Získání seznamu všech dostupných modelů

```python
from llama_stack_client import LlamaStackClient

client = LlamaStackClient(base_url="http://localhost:8321")

print(f"Using Llama Stack version {client._version}")

models = client.models.list()

for model in models:
    print(model)
```

Llama Stack je použit jako běžná knihovna

Získání seznamu všech dostupných modelů

```python
from llama_stack.distribution.library_client import LlamaStackAsLibraryClient

client = LlamaStackAsLibraryClient("run.yaml")
client.initialize()

print(f"Using Llama Stack version {client._version}")

models = client.models.list()

for model in models:
    print(model)
```

Komunikace s LLM

```python
from llama_stack_client import LlamaStackClient

client = LlamaStackClient(base_url="http://localhost:8321")

print(f"Using Llama Stack version {client._version}")

models = client.models.list()
model_id = models[0].identifier

print(f"Using model {model_id}")
```

```
response = client.chat.completions.create(
    model=model_id,
    messages=[{"role": "user", "content": "What is the capital of France?"}]
)

print(response.to_json())
```

Vývoj Llama Stacku

- Změny v API
- Plány na ukončení podpory starších API
  - deprecation
- Náhrada agent API za OpenAI API
- Stabilizace ve verzi 0.3.0 ???

Využití novějšího API

```
from llama_stack_client import LlamaStackClient

client = LlamaStackClient(base_url="http://localhost:8321")

print(f"Using Llama Stack version {client._version}")

models = client.models.list()
model_id = models[0].identifier

print(f"Using model {model_id}")

response = client.chat.completions.create(
    model=model_id,
    messages=[{"role": "user", "content": "What is the capital of France?"}]
)

print(response.to_json())
```

Získávání informací z poskytnutých dokumentů

- RAG
- Ovšem i další vstupy
  - sémantické vyhledávání
  - fulltext vyhledávání
  - hybridní vyhledávání

Prvotní zpracování dokumentů

- připojení k Llama Stacku
- vytvoření nové vektorové databáze
- inicializace vektorové databáze

```
import uuid
from pathlib import Path
from llama_stack_client import LlamaStackClient

client = LlamaStackClient(base_url="http://localhost:8321")
print(f"Using Llama Stack version {client._version}")

vector_store_name= f"vec_{str(uuid.uuid4())[0:8]}"
print(f"Vector store name: {vector_store_name}")

vector_store = client.vector_stores.create(name=vector_store_name)
vector_store_id = vector_store.id

print(f"Vector store ID: {vector_store_id}")
```

- nyní již můžeme do databáze přidat obsah textového souboru nebo jiného dokumentu

```
path=Path("cesta_k_souboru.md")
print(f"File path: {path}")

file_create_response = client.files.create(file=path, purpose="assistants")
print(f"File create response: {file_create_response}")

file_ingest_response = client.vector_stores.files.create(
    vector_store_id=vector_store_id,
    file_id=file_create_response.id,
)
print(f"File ingest response: {file_ingest_response}")
```

- textový dokument je v této chvíli již zpracován
- získání odpovědi z dokumentu

```
models = client.models.list()
model_id = models[0].identifier

print(f"Using model {model_id}")

MODEL_ID="openai/gpt-4-turbo"
```

pomocná funkce pro tisk celé odpovědi

```
def print_rag_response(response):
    print(f"ID: {response.id}")
    print(f"Status: {response.status}")
    print(f"Model: {response.model}")
    print(f"Created at: {response.created_at}")
    print(f"Output items: {len(response.output)}")

    for i, output_item in enumerate(response.output):
        if len(response.output) > 1:
            print(f"\n--- Output Item {i+1} ---")
        print(f"Output type: {output_item.type}")

        if output_item.type in ("text", "message"):
            print(f"Response content: {output_item.content[0].text}")
        elif output_item.type == "file_search_call":
```

získání odpovědi s využitím vektorové databáze

```
            print(f"  Tool Call ID: {output_item.id}")
            print(f"  Tool Status: {output_item.status}")
            print(f"  Queries: {', '.join(output_item.queries)}")
            print(f"  Results: {output_item.results if output_item.results else 'None'}
        else:
            print(f"Response content: {output_item.content}")
```

```
response = client.responses.create(
    model=MODEL_ID,
    input="zadaný dotaz",
    tools=[
        {
            "type": "file_search",
            "vector_store_ids": [vector_store_id],
        }
    ]
)

print_rag_response(response)
```

Ukázka 1/2

- Dokument:
  - `README.TXT` ze hry Supaplex
- Relevantní část z dokumentu:

```
[a round rock] - ZONK
        This is another very common obstacle (and usually a very
        unpleasant one) a Zonk tends to fall down whenever possible
        (i.e. there is a void underneath). Be careful, when a Zonk
        falls on you (Murphy) you will explode (read: die). Murphy can
        push Zonks to the side (not up and down) if nothing is blocking
        the Zonk (i.e. there is a void on the other side). Murphy can
        only push one Zonk at a time, so watch out when dropping Zonks
        next to each other! With some good timing, you can also drop
        Zonks on Electrons (see below) or Snik-snaks (see below) which
        will explode (and not bother you again). Zonks have the nasty
        habit of falling SIDEWAYS off Ram-Chips (see below) and other
        Zonks if possible (i.e. there is a void to the side), on all
        other objects the Zonks will lie steady.
```

Ukázka 2/2

- Dotaz:

```
What is Zonk?
```

- Odpověď:

```
"ZONK" refers to an element in the videogame Supaplex. In the
game, a Zonk is depicted as a round rock, and it is a common and typically
unpleasant obstacle. Zonks tend to fall downwards whenever possible, for
instance, if there is a void directly underneath them. Players must be cautious
as a Zonk falling on the character (Murphy) results in an explosion and
game-over for the player. Murphy can push Zonks laterally (but not vertically)
if there is space for them to move, but can only push one Zonk at a time. Good
timing is needed to use Zonks strategically, such as dropping them on top of
enemies (like Snik-snaks) to eliminate them through explosions.
```

Současná situace okolo Llama Stacku

- Meta -> vLLM
- Zaměření na kompatibilitu s Responses API (OpenAI)
- Podpora pro agentic flow (ovšem jen základní)
- Postupně se některé další funkce odstraňují (!)
- Výsledkem je nestabilita celé platformy
- Pokud vyvíjíte stabilní projekt, je Llama Stack riziko

LangChain

- začneme jednoduchými příklady, které postupně rozšíříme
- poslední příklad bude odpovídat příkladu z Llama Stacku
- odpovědi
- RAG

Získání odpovědi z LLM

- inicializace rozhraní k modelu poskytovaného přes OpenAI
- poslání dotazu do jazykového modelu
- tisk odpovědi

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model_name="gpt-4o-mini")
response = llm.invoke("Say Hi")
print(response)
```

Objekt typu `response`

- inicializace rozhraní k modelu poskytovaného přes OpenAI
- poslání dotazu do jazykového modelu
- tisk odpovědi v naformátované podobě

```
from pprint import pprint
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model_name="gpt-4o-mini")
response = llm.invoke("Say Hi")
pprint(response)
```

Nastavení "teploty"

- inicializace rozhraní k modelu poskytovaného přes OpenAI
- nastavení parametrů jazykového modelu
- poslání dotazu do jazykového modelu
- tisk odpovědi

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model_name="gpt-4o-mini", temperature=0.7)
response = llm.invoke("Say Hi")
print(response)
```

Koncept šablon

- konstrukce jednoduchého řetězce
- předání parametru do výzvy
- poslání dotazu do jazykového modelu
- tisk odpovědi

```
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

prompt = PromptTemplate.from_template("How to say 'hi' in {language} language?")

llm = ChatOpenAI(model_name="gpt-4o-mini")
chain = LLMChain(prompt=prompt, llm=llm)

response = chain.invoke("Czech")
print(response)
```

Řetězce (kolony)

- jednoduchý řetězec
- nastavení parametrů jazykového modelu
- poslání dotazu do jazykového modelu
- tisk odpovědi

```
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["language"],
    template="How to say 'hi' in {language} language?",
)


llm = ChatOpenAI(model_name="gpt-4o-mini")
chain = LLMChain(prompt=prompt, llm=llm)

response = chain.invoke("Czech")
print(response)
```

Řetězce (kolony)

- jednoduchý řetězec vytvořený operátorem |
- nastavení parametrů jazykového modelu
- poslání dotazu do jazykového modelu
- tisk odpovědi

```
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = PromptTemplate.from_template("How to say 'hi' in {language} language?")

llm = ChatOpenAI(model_name="gpt-4o-mini")

chain = prompt | llm | StrOutputParser()

response = chain.invoke({"language": "Czech"})
print(response)
```

Paměť konverzace

- práce s pamětí, ve které je uložena konverzace
- poslání fakta do jazykového modelu
- získání dvou odpovědí, které závisí na faktech v konverzaci

```
from langchain_openai import ChatOpenAI
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory

llm = ChatOpenAI(model_name="gpt-4o-mini")

memory = ConversationBufferMemory()
chatbot = ConversationChain(llm=llm, memory=memory)

print(chatbot.invoke("Hi, I'm Pavel and I live in Czechia."))
print(chatbot.invoke("What's my name?"))
print(chatbot.invoke("Where I live?"))
```

Základy práce s tokeny

- zjištění počtu tokenů pro zadaný řetězec

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model_name="gpt-4o-mini")
print(llm.get_num_tokens("What is firefox?"))
```

Základy práce s tokeny

- inicializace rozhraní k modelu poskytovaného přes OpenAI
- poslání dotazu do jazykového modelu

```
from langchain_openai import ChatOpenAI
from langchain_core.messages.human import HumanMessage

import pprint

llm = ChatOpenAI(model_name="gpt-4o-mini")
```

- tisk celé struktury odpovědi (nikoli pouze textu)

```
response = llm.generate([[HumanMessage("Say Hi")]])
pprint.pprint(response.llm_output)
```

Zpracování dokumentů - loader textových dokumentů

- inicializace objektu pro načítání textových dokumentů
- načtení a zpracování jednoho textového dokumentu
- tisk obsahu objektu představujícího dokument

```
from langchain_community.document_loaders import TextLoader

DOCUMENT_PATH = "documents/EPL-2.0.txt"

loader = TextLoader(DOCUMENT_PATH)
print(loader)

text_documents= loader.load()

print(f"Number of documents: {len(text_documents)}")

for text_document in text_documents:
    print()
    print("-"*80)
    print()
    print(text_document)
```

Zpracování dokumentů - výpis metadat o zpracovaném dokumentu

- inicializace objektu pro načítání textových dokumentů
- načtení a zpracování jednoho textového dokumentu
- tisk typu dokumentu a metadat
- tisk obsahu objektu představujícího dokument

```
from langchain_community.document_loaders import TextLoader

DOCUMENT_PATH = "documents/EPL-2.0.txt"

loader = TextLoader(DOCUMENT_PATH)
print(loader)

text_documents= loader.load()

print(f"Number of documents: {len(text_documents)}")

for text_document in text_documents:
    print()
    print("-"*80)
    print()
    print("Type:", text_document.type)
    print("Metadata:", text_document.metadata)
    print()
    print(text_document.page_content)
```

Zpracování dokumentů - výpis metadat odlišného dokumentu

- inicializace objektu pro načítání textových dokumentů
- načtení a zpracování jednoho textového dokumentu
- tisk typu dokumentu a metadat
- tisk obsahu objektu představujícího dokument

```
from langchain_community.document_loaders import TextLoader

DOCUMENT_PATH = "documents/assemblery.txt"

loader = TextLoader(DOCUMENT_PATH)
print(loader)

text_documents= loader.load()

print(f"Number of documents: {len(text_documents)}")

for text_document in text_documents:
    print()
    print("-"*80)
    print()
    print("Type:", text_document.type)
    print("Metadata:", text_document.metadata)
    print()
    print(text_document.page_content)
```

Zpracování dokumentů - loader PDF dokumentů

- inicializace objektu pro načítání dokumentů ve formátu PDF
- načtení a zpracování jednoho dokumentu ve formátu PDF
- tisk typu dokumentu a metadat
- tisk obsahu objektu představujícího dokument

```
from langchain_community.document_loaders import PyPDFLoader

DOCUMENT_PATH = "documents/EPL-2.0.pdf"

loader = PyPDFLoader(DOCUMENT_PATH)
print(loader)

pdf_documents= loader.load()

print(f"Number of documents: {len(pdf_documents)}")

for pdf_document in pdf_documents:
    print()
    print("-"*80)
    print()
    print("Type:", pdf_document.type)
    print("Metadata:", pdf_document.metadata)
    print()
    print(pdf_document.page_content)
```

Zpracování dokumentů - loader HTML stránek

- inicializace objektu pro načítání obsahu HTML stránek
- načtení a zpracování jedné HTML stránky
- tisk typu dokumentu a metadat
- tisk obsahu objektu představujícího dokument

```
from langchain_community.document_loaders import WebBaseLoader

DOCUMENT_URL = "https://www.eclipse.org/org/documents/epl-2.0/EPL-2.0.html"

loader = WebBaseLoader(DOCUMENT_URL)
print(loader)

text_documents= loader.load()

print(f"Number of documents: {len(text_documents)}")

for text_document in text_documents:
    print()
    print("-"*80)
    print()
    print("Type:", text_document.type)
    print("Metadata:", text_document.metadata)
    print()
    print(text_document.page_content)
```

Zpracování dokumentů - chunking

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

DOCUMENT_PATH = "documents/EPL-2.0.txt"
```

- inicializace objektu pro načítání textových dokumentů
- načtení a zpracování jednoho textového dokumentu
- rozdělení dokumentu do menších částí
- tisk všech částí dokumentu

```
loader = TextLoader(DOCUMENT_PATH)
print(loader)

text_documents= loader.load()

text_splitter= RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks=text_splitter.split_documents(text_documents)

for chunk in chunks:
    print(chunk.page_content)
    print("-"*50)
```

Zpracování dokumentů - výsledek chunkingu

- inicializace objektu pro načítání textových dokumentů
- načtení a zpracování jednoho textového dokumentu
- rozdělení dokumentu do menších částí
- tisk všech částí dokumentu

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

DOCUMENT_PATH = "documents/assemblery.txt"

loader = TextLoader(DOCUMENT_PATH)
print(loader)
print()

text_documents= loader.load()

text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
splitted=text_splitter.split_documents(text_documents)

for chunk in splitted:
    print(chunk.page_content)
    print("-"*50)
```

Zpracování dokumentů - výsledek chunkingu

- inicializace objektu pro načítání textových dokumentů
- načtení a zpracování jednoho textového dokumentu
- rozdělení dokumentu do menších částí
- tisk všech částí dokumentu
- nastavení velikosti jednotlivých částí

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

DOCUMENT_PATH = "documents/assemblery.txt"

loader = TextLoader(DOCUMENT_PATH)
print(loader)
print()

text_documents= loader.load()

text_splitter = RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=50)
splitted=text_splitter.split_documents(text_documents)

for chunk in splitted:
    print(chunk.page_content)
    print("-"*50)
```

Zpracování dokumentů - větší překryv jednotlivých částí

- inicializace objektu pro načítání textových dokumentů
- načtení a zpracování jednoho textového dokumentu
- rozdělení dokumentu do menších částí
- tisk všech částí dokumentu
- nastavení velikosti jednotlivých částí a velikosti překryvu

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

DOCUMENT_PATH = "documents/assemblery.txt"

loader = TextLoader(DOCUMENT_PATH)
print(loader)
print()

text_documents= loader.load()

text_splitter = RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=100)
splitted=text_splitter.split_documents(text_documents)

for chunk in splitted:
    print(chunk.page_content)
    print("-"*50)
```

Vektorové databáze

- specializované databáze
- optimalizovány na efektivní ukládání vektorů
- optimalizovány na efektivní vyhledávání vektorů
- ne na základě shody, ale podobnosti
- různé definice podobnosti

Vzdálenost vektorů

- Eukleidovská vzdálenost
- kosinus úhlu mezi vektory
- skalární součin pro normalizované vektory
- Manhattanská vzdálenost

- Využití vektorových databází
  - NLP
  - rozpoznání obrázků a hlasů
  - detekce anomálií
  - LLM
- Existující vektorové databáze
  - Aerospike
  - AllegroGraph
  - Apache Cassandra
  - Azure Cosmos DB
  - Chroma

- ClickHouse
- Couchbase
- CrateDB
- DataStax
- Elasticsearch
- HAKES
- HDF5 Query Indexing
- JaguarDB
- LanceDB
- Lantern
- LlamaIndex
- MariaDB
- Marqo
- Meilisearch
- Milvus
- MongoDB Atlas
- Neo4j
- ObjectBox
- OpenSearch
- Oracle Database
- Pinecone
- Pixeltable (Incremental Embedding)
- Postgres with pgvector
- Qdrant
- Redis Stack
- Snowflake
- SurrealDB
- Typesense
- Vespa
- Weaviate

Embedding

- Vnoření slov
  - Převedení slov na vektory
  - Slova s podobnými vlastnostmi se nachází blízko sebe
  - word2vec Tomáš Mikolov
- Vnoření vět
  - dtto, ale pro celé věty nebo části textu
  - to je i důvod, proč jsme prováděli chunking

Faiss

- Facebook AI Similarity Search
- Open source
- Podpora indexů
- Algoritmy pro vyhledávání vektorů
- Algoritmy pro clustering

```
                          y
                          |
                          |
                          |
                          |          [5,5]
        o       o         |      o   o   o
                          |      o   o   o
          o               |      o   o   o
      [-4,3]              |    [3,3]   [5,3]
                          |
-------------------------[0,0]-------------------------
                                                      x
                          |
                          |          o
                          |
                          |      o       o
                          |            [5,-5]
                          |
                          |
                          |
                          |
```

První seznámení s vektory

- konstrukce dvou vektorů se souřadnicemi bodů v rovině
- výpis obsahu obou vektorů na standardní výstup
- konstrukce seznamu souřadnic vytvořených z obou vektorů

```
x = [-5, -4, -3,    3,  4,  5,   3, 3, 3,  4, 4, 4,  5, 5, 5]

y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5,  3, 4, 5,  3, 4, 5]

print(x)
print(y)

print(list(zip(x, y)))
```

Vektory v knihovně NumPy

- konstrukce dvou vektorů se souřadnicemi bodů v rovině
- konstrukce 2D matice se souřadnicemi bodů v rovině z obou

```
import numpy as np

x = [-5, -4, -3,    3,  4,  5,   3, 3, 3,  4, 4, 4,  5, 5, 5]

y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5,  3, 4, 5,  3, 4, 5]

points = np.column_stack((x,y)).astype("float32")
```

vektorů
- výpis obsahu matice

```
print(points)
```

Konstrukce indexu

- konstrukce indexu knihovnou FAISS
- tisk základních informací o vytvořeném indexu

```
import faiss
import numpy as np

x = [-5, -4, -3,    3,  4,  5,   3, 3, 3,  4, 4, 4,  5, 5, 5]

y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5,  3, 4, 5,  3, 4, 5]

points = np.column_stack((x,y)).astype("float32")
print(points)

index = faiss.IndexFlatL2(2)
index.add(points)

print()
print("Dimension(s):         ", index.d)
print("Total values in index:", index.ntotal)
print("Is index trained:     ", index.is_trained)
```

Nalezení vektorů na základě podobnosti

- nalezení nejpodobnějších vektorů knihovnou FAISS
- použití L2 metriky
- výpis indexů nejpodobnějších vektorů

```
import faiss
import numpy as np

x = [-5, -4, -3,    3,  4,  5,   3, 3, 3,  4, 4, 4,  5, 5, 5]

y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5,  3, 4, 5,  3, 4, 5]

points = np.column_stack((x,y)).astype("float32")
print(points)

index = faiss.IndexFlatL2(2)
index.add(points)

print()
print("Dimension(s):         ", index.d)
print("Total values in index:", index.ntotal)
print("Is index trained:     ", index.is_trained)

query_vector = np.array([[3, 3]]).astype("float32")
print(query_vector)

k = len(x)

distances, indices = index.search(query_vector, k)

print("Nearest neighbors:")
print("neighbour  distance  index")
print("--------------------------")
for i in range(k):
    print(f"{i+1:3}      {distances[0][i]:5}       {indices[0][i]:2}")
```

```
Nearest neighbors:
neighbour  distance  index
--------------------------
  1        0.0        6
  2        1.0        7
  3        1.0        9
  4        2.0       10
  5        4.0        8
  6        4.0       12
  7        5.0       11
  8        5.0       13
  9        8.0       14
 10       37.0        4
 11       40.0        2
 12       49.0        1
 13       64.0        3
 14       68.0        0
 15       68.0        5
```

Nalezení vektorů na základě podobnosti

- nalezení nejpodobnějších vektorů knihovnou FAISS
- použití L2 metriky
- výpis souřadnic nejpodobnějších vektorů

```
import faiss
import numpy as np

x = [-5, -4, -3,    3,  4,  5,   3, 3, 3,  4, 4, 4,  5, 5, 5]

y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5,  3, 4, 5,  3, 4, 5]

points = np.column_stack((x,y)).astype("float32")
print(points)

index = faiss.IndexFlatL2(2)
index.add(points)

print()
print("Dimension(s):         ", index.d)
print("Total values in index:", index.ntotal)
print("Is index trained:     ", index.is_trained)

query_vector = np.array([[3, 3]]).astype("float32")
print(query_vector)

k = len(x)

distances, indices = index.search(query_vector, k)

print("Nearest neighbors:")
print("neighbour  distance  coordinates  ")
print("----------------------------------")
for i in range(k):
    print(f"{i+1:3}      {distances[0][i]:5}       {points[indices[0][i]]}")
```

```
Nearest neighbors:
neighbour  distance  coordinates
```

```
----------------------------------
  1        0.0       [3. 3.]
  2        1.0       [3. 4.]
  3        1.0       [4. 3.]
  4        2.0       [4. 4.]
  5        4.0       [3. 5.]
  6        4.0       [5. 3.]
  7        5.0       [4. 5.]
  8        5.0       [5. 4.]
  9        8.0       [5. 5.]
 10       37.0       [ 4. -3.]
 11       40.0       [-3.  5.]
 12       49.0       [-4.  3.]
 13       64.0       [ 3. -5.]
 14       68.0       [-5.  5.]
 15       68.0       [ 5. -5.]
```

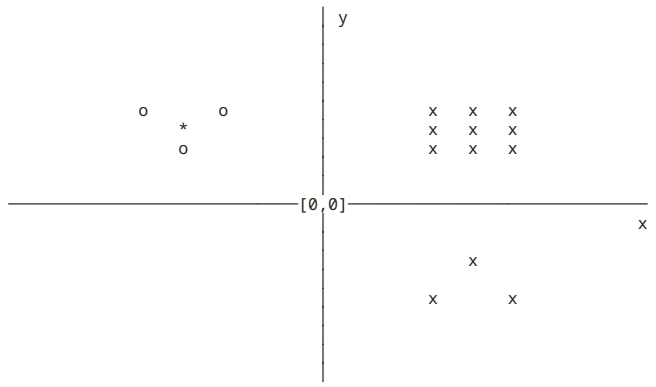
Nejbližší tři body k bodu [-4, 4]

```
query_vector = np.array([[-4, 4]]).astype("float32")
print(query_vector)

k = 3
distances, indices = index.search(query_vector, k)
```

```
Nearest neighbors:
neighbour  distance  coordinates
----------------------------------
  1        1.0       [-4.  3.]
  2        2.0       [-5.  5.]
  3        2.0       [-3.  5.]
```

Vizualizace výsledků



Nalezení nejpodobnějších vektorů knihovnou FAISS

- použití metriky založené na skalárním součinu
- výpis souřadnic nejpodobnějších vektorů
- vektory nejsou normalizovány

```
import faiss
import numpy as np

x = [-5, -4, -3,    3,  4,  5,   3, 3, 3,  4, 4, 4,  5, 5, 5]

y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5,  3, 4, 5,  3, 4, 5]

points = np.column_stack((x,y)).astype("float32")
print(points)

index = faiss.IndexFlatIP(2)
index.add(points)

print()
print("Dimension(s):         ", index.d)
print("Total values in index:", index.ntotal)
print("Is index trained:     ", index.is_trained)

query_vector = np.array([[3, 3]]).astype("float32")
print(query_vector)

k = len(x)

distances, indices = index.search(query_vector, k)

print("Nearest neighbors:")
print("neighbour  distance  coordinates  ")
print("----------------------------------")
for i in range(k):
    print(f"{i+1:3}      {distances[0][i]:5}       {points[indices[0][i]]}")
```

Nalezení nejpodobnějších vektorů knihovnou FAISS

- použití metriky založené na skalárním součinu
- výpis souřadnic nejpodobnějších vektorů
- vektory jsou normalizovány

```
import faiss
import numpy as np

x = [-5, -4, -3,    3,  4,  5,   3, 3, 3,  4, 4, 4,  5, 5, 5]

y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5,  3, 4, 5,  3, 4, 5]

points = np.column_stack((x,y)).astype("float32")
print(points)

for i in range(len(points)):
   vector = points[i]
   normalized = np.linalg.norm(vector)
   vector /= normalized
   points[i] = vector

print()
print("Normalized:")
print(points)
```

```
index = faiss.IndexFlatIP(2)
index.add(points)

print()
print("Dimension(s):         ", index.d)
print("Total values in index:", index.ntotal)
print("Is index trained:     ", index.is_trained)

query_vector = np.array([[3, 3]]).astype("float32")
normalized = np.linalg.norm(query_vector)
query_vector /= normalized
print(query_vector)

k = len(x)

distances, indices = index.search(query_vector, k)

print("Nearest neighbors:")
print("neighbour  distance  coordinates  ")
print("----------------------------------")
for i in range(k):
    print(f"{i+1:3}      {distances[0][i]:+7.4f}     {points[indices[0][i]]}")
```

Benchmark

- benchmark rychlosti nalezení nejpodobnějších vektorů
- výpis výsledků v tabulkové formě
- vizualizace výsledků formou grafu

Nalezeni k nejblizsich vektoru v mnozine n vektoru.

```
from time import time

import faiss
import matplotlib.pyplot as plt
import numpy as np



def similarity_search(n, k):


    DIMENSIONS=128

    t1 = time()

    data = np.random.rand(n, 128).astype('float32')

    t2 = time()

    index = faiss.IndexFlatL2(DIMENSIONS)
    index.add(data)

    t3 = time()

    query_vector = np.random.rand(1, DIMENSIONS).astype("float32")

    distances, indices = index.search(query_vector, k)
    t4 = time()

    assert len(distances) == k
    assert len(indices) == k

    return n, t2-t1, t3-t2, t4-t3


ns = []
ts_rand = []
ts_index = []
ts_search = []

for n in np.linspace(1000000, 10000000, 10):
    print(n)
    n, t_rand, t_index, t_search = similarity_search(int(n), 1)
    ns.append(n)
    ts_rand.append(t_rand)
    ts_index.append(t_index)
    ts_search.append(t_search)


plt.plot(ns, ts_rand, "r-", label="numpy.random.rand")
plt.plot(ns, ts_index, "b-", label="index creation")
plt.plot(ns, ts_search, "m-", label="similarity search")

plt.legend(loc="upper left")

plt.grid(True)

plt.savefig("faiss_benchmark_2.png")

plt.show()
```

Vizualizace vektorů v rovině

- vizualizace koncových bodů vektorů v rovině

```
import matplotlib.pyplot as plt
import numpy as np

DIMENSIONS=2

N=1000

K=100

vectors = np.random.rand(N, DIMENSIONS).astype("float32")

plt.figure(figsize=(8, 8), dpi=80)

plt.plot(vectors[:,0], vectors[:,1], "+k", label="original vectors", markersize=5)

plt.legend(loc="upper left")

plt.grid(True)

plt.savefig("faiss-9.png")

plt.show()
```

Vizualizace nejpodobnějších vektorů v rovině

- vykreslení nejpodobnějších vektorů získaných na základě L2 metriky
- vektory nejsou normalizovány

```
import faiss
import matplotlib.pyplot as plt
import numpy as np

DIMENSIONS=2

N=1000

K=100

vectors = np.random.rand(N, DIMENSIONS).astype("float32")

index = faiss.IndexFlatL2(DIMENSIONS)
index.add(vectors)

query_vector = np.array([[0.5, 0.5]]).astype("float32")

distances, indices = index.search(query_vector, K)

plt.figure(figsize=(8, 8), dpi=80)

plt.plot(vectors[:,0], vectors[:,1], "+k", label="original vectors", markersize=5)

xs = vectors[:,0][indices][0]
ys = vectors[:,1][indices][0]
plt.plot(xs, ys, "+r", label="nearest vectors", markersize=5)

x = query_vector[0][0]
y = query_vector[0][1]
plt.plot(x, y, "ob", label="query vector", markersize=10)

plt.legend(loc="upper left")

plt.grid(True)

plt.savefig("faiss-A.png")

plt.show()
```

Vizualizace nejpodobnějších vektorů v rovině

- vykreslení nejpodobnějších vektorů získaných na základě skalárního součinu
- vektory nejsou normalizovány

```
import faiss
import matplotlib.pyplot as plt
import numpy as np

DIMENSIONS=2

N=1000

K=100

vectors = np.random.rand(N, DIMENSIONS).astype("float32")

index = faiss.IndexFlatIP(DIMENSIONS)
index.add(vectors)

query_vector = np.array([[0.5, 0.5]]).astype("float32")

distances, indices = index.search(query_vector, K)

plt.figure(figsize=(8, 8), dpi=80)

plt.plot(vectors[:,0], vectors[:,1], "+k", label="original vectors", markersize=5)

xs = vectors[:,0][indices][0]
ys = vectors[:,1][indices][0]
plt.plot(xs, ys, "+r", label="nearest vectors", markersize=5)

x = query_vector[0][0]
y = query_vector[0][1]
plt.plot(x, y, "ob", label="query vector", markersize=10)

plt.legend(loc="upper left")

plt.grid(True)

plt.savefig("faiss-B.png")

plt.show()
```

Vizualizace nejpodobnějších vektorů v rovině

- vykreslení nejpodobnějších vektorů získaných na základě skalárního součinu
- vektory jsou normalizovány

```
import faiss
import matplotlib.pyplot as plt
import numpy as np

DIMENSIONS=2

N=1000

K=100

vectors = np.random.rand(N, DIMENSIONS).astype("float32")

for i in range(len(vectors)):
   vector = vectors[i]
   normalized = np.linalg.norm(vector)
   vector /= normalized
   vectors[i] = vector

index = faiss.IndexFlatIP(DIMENSIONS)
index.add(vectors)

query_vector = np.array([[0.5, 0.5]])
normalized = np.linalg.norm(query_vector)
query_vector /= normalized

distances, indices = index.search(query_vector, K)

plt.figure(figsize=(8, 8), dpi=80)
```

```
plt.plot(vectors[:,0], vectors[:,1], "+k", label="original vectors", markersize=5)

xs = vectors[:,0][indices][0]
ys = vectors[:,1][indices][0]
plt.plot(xs, ys, "+r", label="nearest vectors", markersize=5)

x = query_vector[0][0]
y = query_vector[0][1]
plt.plot(x, y, "ob", label="query vector", markersize=10)

plt.legend(loc="upper left")

plt.grid(True)

plt.savefig("faiss-C.png")

plt.show()
```

Vizualizace nejpodobnějších vektorů v rovině

- vykreslení nejpodobnějších vektorů získaných na základě skalárního součinu
- vektory jsou normalizovány
- vykresleny jsou ovšem původní vektory

```
import faiss
import matplotlib.pyplot as plt
import numpy as np

DIMENSIONS=2

N=1000

K=100

vectors = np.random.rand(N, DIMENSIONS).astype("float32")

normalized = np.matrix.copy(vectors)

for i in range(len(vectors)):
   vector = vectors[i]
   norm = np.linalg.norm(vector)
   normalized[i] = vector / norm

index = faiss.IndexFlatIP(DIMENSIONS)
index.add(normalized)

query_vector = np.array([[0.5, 0.5]])
norm = np.linalg.norm(query_vector)
normalized_query_vector = query_vector / norm

distances, indices = index.search(normalized_query_vector, K)

plt.figure(figsize=(8, 8), dpi=80)

plt.plot(vectors[:,0], vectors[:,1], "+k", label="original vectors", markersize=5)

xs = vectors[:,0][indices][0]
ys = vectors[:,1][indices][0]
plt.plot(xs, ys, "+r", label="nearest vectors", markersize=5)

x = query_vector[0][0]
y = query_vector[0][1]
plt.plot(x, y, "ob", label="query vector", markersize=10)

plt.legend(loc="upper left")

plt.grid(True)

plt.savefig("faiss-D.png")

plt.show()
```

Vizualizace vektorů v rovině

- vyhledání a vykreslení nejvíce NEpodobných vektorů
- vykreslení těchto vektorů
- složky všech vektorů jsou typu float32

```
import faiss
import matplotlib.pyplot as plt
import numpy as np

DIMENSIONS=2

N=1000

K=100

vectors = np.random.rand(N, DIMENSIONS).astype("float32")

normalized = np.matrix.copy(vectors)

for i in range(len(vectors)):
   vector = vectors[i]
   norm = np.linalg.norm(vector)
   normalized[i] = vector / norm

index = faiss.IndexFlatIP(DIMENSIONS)
index.add(normalized)

query_vector = np.array([[0.5, 0.5]])
norm = np.linalg.norm(query_vector)
normalized_query_vector = query_vector / norm

distances, indices = index.search(-normalized_query_vector, K)

plt.figure(figsize=(8, 8), dpi=80)

plt.plot(vectors[:,0], vectors[:,1], "+k", label="original vectors", markersize=5)

xs = vectors[:,0][indices][0]
ys = vectors[:,1][indices][0]
plt.plot(xs, ys, "+r", label="nearest vectors", markersize=5)

x = query_vector[0][0]
y = query_vector[0][1]
plt.plot(x, y, "ob", label="query vector", markersize=10)

plt.legend(loc="upper left")
```

```
plt.grid(True)

plt.savefig("faiss-G.png")

plt.show()
```

Vizualizace nejpodobnějších vektorů v rovině

- vykreslení nejpodobnějších vektorů získaných na základě jejich vzdálenosti
- vykresleny jsou původní vektory
- složky všech vektorů jsou typu float16

```
import faiss
import matplotlib.pyplot as plt
import numpy as np

DIMENSIONS=2

N=1000

K=100

vectors = np.random.rand(N, DIMENSIONS).astype("float16")

normalized = np.matrix.copy(vectors)

for i in range(len(vectors)):
   vector = vectors[i]
   norm = np.linalg.norm(vector)
   normalized[i] = vector / norm

index = faiss.IndexFlatL2(DIMENSIONS)
index.add(normalized)

query_vector = np.array([[0.5, 0.5]]).astype("float16")
norm = np.linalg.norm(query_vector)
normalized_query_vector = query_vector / norm

distances, indices = index.search(normalized_query_vector, K)

plt.figure(figsize=(8, 8), dpi=80)

plt.plot(vectors[:,0], vectors[:,1], "+k", label="original vectors", markersize=5)

xs = vectors[:,0][indices][0]
ys = vectors[:,1][indices][0]
plt.plot(xs, ys, "+r", label="nearest vectors", markersize=5)

x = query_vector[0][0]
y = query_vector[0][1]
plt.plot(x, y, "ob", label="query vector", markersize=10)

plt.legend(loc="upper left")

plt.grid(True)

plt.savefig("faiss-H.png")

plt.show()
```

Benchmark rychlosti nalezení nejpodobnějších vektorů

- vizualizace výsledků formou grafu
- porovnání float16 a float32

Nalezeni k nejblizsich vektoru v mnozine n vektoru.

```
from time import time

import faiss
import matplotlib.pyplot as plt
import numpy as np


def similarity_search(n, k, float_type):


    DIMENSIONS=128

    data = np.random.rand(n, 128).astype(float_type)

    index = faiss.IndexFlatL2(DIMENSIONS)
    index.add(data)

    t1 = time()

    query_vector = np.random.rand(1, DIMENSIONS).astype(float_type)

    distances, indices = index.search(query_vector, k)
    t2 = time()

    return n, t2-t1


def benchmark(from_n, to_n, steps, float_type):
    ns = []
    ts_search = []

    for n in np.linspace(from_n, to_n, steps):
        print(n)
        n, t_search = similarity_search(int(n), 1, float_type)
        ns.append(n)
        ts_search.append(t_search)

    return ns, ts_search


from_n = 1000000
to_n = 10000000
steps = 10

ns, float16_times = benchmark(from_n, to_n, steps, "float16")
_, float32_times = benchmark(from_n, to_n, steps, "float32")

plt.plot(ns, float16_times, "r-", label="float16")
plt.plot(ns, float32_times, "b-", label="float32")

plt.legend(loc="upper left")

plt.grid(True)
```

```
plt.savefig("faiss_benchmark_3.png")

plt.show()
```

PgVector

- Rozšíření databáze PostgreSQL
- Nový typ "vektor"
- Podpora pro různé typy prvků vektorů
- Nové operátory (vzdálenosti vektorů)
  - různé metriky

```
Operátor  Stručný popis
-------------------------
<->   Eukleidovská vzdálenost
<#>   skalární součin
<=>   1 - kosinus úhlu mezi vektory
<+>   Manhattanská metrika
<~>   Hammingova vzdálenost
<%>   Jaccardův index
```

Konstrukce tabulky s dvouprvkovými vektory

```python
import psycopg2

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

print(connection)

CREATE_TABLE_STATEMENT = """
    CREATE TABLE IF NOT EXISTS v2 (
        id bigserial PRIMARY KEY,
        embedding vector(2) NOT NULL
    );
"""

LIST_TABLES_QUERY = """
    SELECT table_schema,table_name
      FROM information_schema.tables
     WHERE table_schema='public'
     ORDER BY table_schema,table_name;
"""

with connection.cursor() as cursor:
    print(CREATE_TABLE_STATEMENT)
    cursor.execute(CREATE_TABLE_STATEMENT)
    connection.commit()

    print(LIST_TABLES_QUERY)
    cursor.execute(LIST_TABLES_QUERY)
    tables = cursor.fetchall()

    for table in tables:
        print(table)
```

Konstrukce tabulky s delšími vektory

```python
import psycopg2

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

print(connection)

CREATE_TABLE_STATEMENT = """
    CREATE TABLE IF NOT EXISTS v384 (
        id bigserial PRIMARY KEY,
        embedding vector(384) NOT NULL
    );
"""

LIST_TABLES_QUERY = """
    SELECT table_schema,table_name
      FROM information_schema.tables
     WHERE table_schema='public'
     ORDER BY table_schema,table_name;
"""

with connection.cursor() as cursor:
    print(CREATE_TABLE_STATEMENT)
    cursor.execute(CREATE_TABLE_STATEMENT)
    connection.commit()

    print(LIST_TABLES_QUERY)
    cursor.execute(LIST_TABLES_QUERY)
    tables = cursor.fetchall()

    for table in tables:
        print(table)
```

Konstrukce tabulky s původními texty

```python
import psycopg2

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

print(connection)

CREATE_TABLE_STATEMENT = """
```

```
    CREATE TABLE IF NOT EXISTS v384b (
        id bigserial PRIMARY KEY,
        embedding vector(384) NOT NULL,
        sentence TEXT NOT NULL
    );
"""

LIST_TABLES_QUERY = """
    SELECT table_schema,table_name
      FROM information_schema.tables
     WHERE table_schema='public'
     ORDER BY table_schema,table_name;
"""

with connection.cursor() as cursor:
    print(CREATE_TABLE_STATEMENT)
    cursor.execute(CREATE_TABLE_STATEMENT)
    connection.commit()

    print(LIST_TABLES_QUERY)
    cursor.execute(LIST_TABLES_QUERY)
    tables = cursor.fetchall()

    for table in tables:
        print(table)
```

Naplnění tabulky

```
import psycopg2

import numpy as np
from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

register_vector(connection)

x = [-5, -4, -3,    3,  4,  5,   3, 3, 3, 4, 4, 4, 5, 5, 5]
y = [ 5,  3,  5,   -5, -3, -5,   3, 4, 5, 3, 4, 5, 3, 4, 5]

with connection.cursor() as cursor:
    for i in range(len(x)):
        vector = np.array([x[i], y[i]])
        print(type(vector), vector)
        cursor.execute("INSERT INTO v2 (embedding) VALUES (%s)", (vector, ))
    connection.commit()
```

Výběr na základě vzdálenosti

```
import psycopg2

from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

register_vector(connection)

with connection.cursor() as cursor:
    cursor.execute("SELECT embedding FROM v2 ORDER BY embedding <-> %s::vector LIMIT 5'
    records = cursor.fetchall()
    for record in records:
        print(record[0])
```

Výběr na základě vzdálenosti

```
import psycopg2

from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

register_vector(connection)

query = """
    SELECT embedding
      FROM v2
     WHERE embedding <-> %s::vector < %s
"""

for distance in range(0, 10):
    print("Distance:", distance)
    with connection.cursor() as cursor:
        cursor.execute(query, ([3,3], distance))
        records = cursor.fetchall()
        for record in records:
            print(record[0])
    print("-"*50)
```

Další operátory

```
import psycopg2

from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

register_vector(connection)


def list_by_distance(title, query):
    print(title)
    print("-" * 70)
    with connection.cursor() as cursor:
        cursor.execute(query, ([3,3], ))
        records = cursor.fetchall()
        for record in records:
            print(record[0], record[1])
```

```
    print()


query_l2_distance = """
        SELECT embedding, embedding <-> %s::vector as distance
          FROM v2
         ORDER BY distance
    """


query_cosine_distance = """
        SELECT embedding, embedding <=> %s::vector as distance
          FROM v2
         ORDER BY distance
    """


query_inner_product_distance = """
        SELECT embedding, (embedding <#> %s::vector) * -1 as distance
          FROM v2
         ORDER BY distance
    """


list_by_distance("L2", query_l2_distance)
list_by_distance("cosine", query_cosine_distance)
list_by_distance("inner product", query_inner_product_distance)
```

Tabulka s binárními vektory

```
import psycopg2

connection = psycopg2.connect(
    host="localhost", port=54321, user="tester", password="123qwe", dbname="test"
)

print(connection)

CREATE_TABLE_STATEMENT = """
    CREATE TABLE IF NOT EXISTS b1 (
        id bigserial PRIMARY KEY,
        embedding bit (4) NOT NULL
    );
"""

LIST_TABLES_QUERY = """
    SELECT table_schema,table_name
      FROM information_schema.tables
     WHERE table_schema='public'
     ORDER BY table_schema,table_name;
"""

with connection.cursor() as cursor:
    print(CREATE_TABLE_STATEMENT)
    cursor.execute(CREATE_TABLE_STATEMENT)
    connection.commit()

    print(LIST_TABLES_QUERY)
    cursor.execute(LIST_TABLES_QUERY)
    tables = cursor.fetchall()

    for table in tables:
        print(table)
```

Zápis dat do tabulky s binárními vektory

```
import psycopg2

import numpy as np
from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="localhost", port=54321, user="tester", password="123qwe", dbname="test"
)

register_vector(connection)

vectors = [
        "0000",
        "1111",
        "1100",
        "0011",
        ]

with connection.cursor() as cursor:
    for vector in vectors:
        cursor.execute("INSERT INTO b1 (embedding) VALUES (%s)", (vector, ))
    connection.commit()
```

Výběrové operátory pro tabulku s binárními vektory

```
import psycopg2

from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="localhost", port=54321, user="tester", password="123qwe", dbname="test"
)

register_vector(connection)

with connection.cursor() as cursor:
    cursor.execute("SELECT embedding FROM b1 ORDER BY embedding <~> %s::bit(4) LIMIT 5'
    records = cursor.fetchall()
    for record in records:
        print(record[0])
```

Výběrové operátory pro tabulku s binárními vektory

```
import psycopg2

from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="localhost", port=54321, user="tester", password="123qwe", dbname="test"
```

```
)

register_vector(connection)

with connection.cursor() as cursor:
    cursor.execute("SELECT embedding, embedding <~> '0000' AS distance FROM b1 ORDER B
    records = cursor.fetchall()
    for record in records:
        print(record[0], record[1])
```

SentenceTransformer

Inicializace embedding modelu

- zadaný model se poměrně často používá v praxi

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)
```

Inicializace embedding modelu

- je použit odlišný model, než v předchozím příkladu

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-mpnet-base-v2")

print(model)
```

Existují i české modely

- používají se naprosto stejným způsobem, jako modely anglické

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("Seznam/small-e-czech")

print(model)
```

Vektorizace vět zapsaných v angličtině

- výsledné vektory pochopitelně závisí na použitém modelu

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)

sentences = [
    "The rain in Spain falls mainly on the plain",
    "The tesselated polygon is a special type of polygon",
    "The quick brown fox jumps over the lazy dog",
    "To be or not to be, that is the question",
    "It is a truth universally acknowledged...",
    "The goat ran down the hill"
]

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

print(f"Data type: {type(embeddings)}")

print(embeddings)

np.info(embeddings)
```

Zjištění míry podobnosti zapsaných vět

- poněkud umělý příklad, který ale ukazuje možnosti embedded modelů

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)

sentences = [
    "The rain in Spain falls mainly on the plain",
    "The tesselated polygon is a special type of polygon",
    "The quick brown fox jumps over the lazy dog",
    "To be or not to be, that is the question",
    "It is a truth universally acknowledged...",
    "The goat ran down the hill"
]

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

print(embeddings)
similarities = model.similarity(embeddings, embeddings)
print(similarities)
```

Vyhledávání ve vektorizovaném textu

- podobnostní vyhledávání
- použití indexu FlatL2
- ke každé vstupní větě se vrátí tři nejpodobnější věty z vektorové databáze

```
from sentence_transformers import SentenceTransformer

import faiss

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)

sentences = [
    "The rain in Spain falls mainly on the plain",
    "The tesselated polygon is a special type of polygon",
    "The quick brown fox jumps over the lazy dog",
    "To be or not to be, that is the question",
    "It is a truth universally acknowledged...",
    "The goat ran down the hill"
]

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

similarities = model.similarity(embeddings, embeddings)

DIMENSIONS = embeddings.shape[1]

index = faiss.IndexFlatL2(DIMENSIONS)
index.add(embeddings)
```

```
print(f"Index: {index.ntotal}")


def find_similar_sentences(query_sentence, k):
    query_embedding = model.encode([query_sentence])
    distances, indices = index.search(query_embedding, k)
    print("-"*40)
    print(f"Query: {query_sentence}")
    print(f"Most {k} similar sentences:")
    for i, idx in enumerate(indices[0]):
        print(f"{i + 1}: {sentences[idx]} (Distance: {distances[0][i]})")


find_similar_sentences("The quick brown fox jumps over the lazy dog", 3)
find_similar_sentences("quick brown fox jumps over lazy dog", 3)
find_similar_sentences("The quick brown fox jumps over the angry dog", 3)
find_similar_sentences("The quick brown cat jumps over the lazy dog", 3)
```

Sémantické vyhledávání

- povšimněte si, že vyhledávané věty se podobají větám z databáze "jen" sémanticky, ne textově

```
from sentence_transformers import SentenceTransformer

import faiss

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)

sentences = [
    "The rain in Spain falls mainly on the plain",
    "The tesselated polygon is a special type of polygon",
    "The quick brown fox jumps over the lazy dog",
    "To be or not to be, that is the question",
    "It is a truth universally acknowledged...",
    "The goat ran down the hill"
]

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

similarities = model.similarity(embeddings, embeddings)

DIMENSIONS = embeddings.shape[1]

index = faiss.IndexFlatL2(DIMENSIONS)
index.add(embeddings)

print(f"Index: {index.ntotal}")


def find_similar_sentences(query_sentence, k):
    query_embedding = model.encode([query_sentence])
    distances, indices = index.search(query_embedding, k)
    print("-"*40)
    print(f"Query: {query_sentence}")
    print(f"Most {k} similar sentences:")
    for i, idx in enumerate(indices[0]):
        print(f"{i + 1}: {sentences[idx]} (Distance: {distances[0][i]})")


find_similar_sentences("The rain in Spain falls mainly on the plain", 3)
find_similar_sentences("The rain in Czechia falls mainly on the plain", 3)
find_similar_sentences("rainy weather in Spain, especially on plains", 3)
```

Sémantické vyhledávání

- extrémní případ: vyhledávaná věta se textově zcela odlišuje od všech vět v databázi
- ovšem je zde sémantická shoda

```
from sentence_transformers import SentenceTransformer

import faiss

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)

sentences = [
    "The rain in Spain falls mainly on the plain",
    "The tesselated polygon is a special type of polygon",
    "The quick brown fox jumps over the lazy dog",
    "To be or not to be, that is the question",
    "It is a truth universally acknowledged...",
    "How old are you?",
    "The goat ran down the hill"
]

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

similarities = model.similarity(embeddings, embeddings)

DIMENSIONS = embeddings.shape[1]

index = faiss.IndexFlatL2(DIMENSIONS)
index.add(embeddings)

print(f"Index: {index.ntotal}")


def find_similar_sentences(query_sentence, k):
    query_embedding = model.encode([query_sentence])
    distances, indices = index.search(query_embedding, k)
    print("-"*40)
    print(f"Query: {query_sentence}")
    print(f"Most {k} similar sentences:")
    for i, idx in enumerate(indices[0]):
        print(f"{i + 1}: {sentences[idx]} (Distance: {distances[0][i]})")


find_similar_sentences("What is your age?", 3)
```

Sémantické vyhledávání

```
from sentence_transformers import SentenceTransformer
```

- další ukázky sémantického vyhledávání
- vyhledávání na základě jediného slova

```python
import faiss

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)

sentences = [
    "The rain in Spain falls mainly on the plain",
    "The tesselated polygon is a special type of polygon",
    "The quick brown fox jumps over the lazy dog",
    "To be or not to be, that is the question",
    "It is a truth universally acknowledged...",
    "How old are you?",
    "The goat ran down the hill"
]

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

similarities = model.similarity(embeddings, embeddings)

DIMENSIONS = embeddings.shape[1]

index = faiss.IndexFlatL2(DIMENSIONS)
index.add(embeddings)

print(f"Index: {index.ntotal}")


def find_similar_sentences(query_sentence, k):
    query_embedding = model.encode([query_sentence])
    distances, indices = index.search(query_embedding, k)
    print("-"*40)
    print(f"Query: {query_sentence}")
    print(f"Most {k} similar sentences:")
    for i, idx in enumerate(indices[0]):
        print(f"{i + 1}: {sentences[idx]} (Distance: {distances[0][i]})")


find_similar_sentences("Shakespeare", 3)
find_similar_sentences("animal", 3)
find_similar_sentences("geometry", 3)
find_similar_sentences("weather", 3)
```

Sémantické vyhledávání v již vytvořeném modelu

- nyní provedeme vyhledávání nikoli v naší malé databázi
- ale v celé trénovací sadě

```python
from datasets import load_dataset
from sentence_transformers import SentenceTransformer

import faiss

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")
print(model)

dataset = load_dataset("sentence-transformers/wikipedia-en-sentences", split="train")
print(dataset)

sentences = [sentence for sentence in dataset["sentence"][0:1000]]
print(f"{len(sentences)} sentences created")

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

DIMENSIONS = embeddings.shape[1]
index = faiss.IndexFlatL2(DIMENSIONS)
index.add(embeddings)
print(f"Index: {index.ntotal}")


def find_similar_sentences(query_sentence, k):
    query_embedding = model.encode([query_sentence])
    distances, indices = index.search(query_embedding, k)
    print("-"*40)
    print(f"Query: {query_sentence}")
    print(f"Most {k} similar sentences:")
    for i, idx in enumerate(indices[0]):
        print(f"{i + 1}: {sentences[idx]} (Distance: {distances[0][i]})")


find_similar_sentences("city", 3)
find_similar_sentences("animal", 3)
find_similar_sentences("geometry", 3)
find_similar_sentences("weather", 3)
find_similar_sentences("game", 3)
find_similar_sentences("school", 3)
```

Strukturovaná podoba příkladu pro využití embedded modelu

- opět založeno na datové sadě s příklady anglických textů

```python
from datasets import load_dataset
from sentence_transformers import SentenceTransformer

import faiss

MODEL_NAME = "paraphrase-MiniLM-L6-v2"
DATASET_ID = "sentence-transformers/wikipedia-en-sentences"


def initialize_model(model_name):
    print("Model initialization started")
    model = SentenceTransformer(model_name)
    print(model)
    print("Model initialization finished")
    return model


def load_dataset_by_id(dataset_id):
    print("Loading dataset started")
    dataset = load_dataset(dataset_id, split="train")
    print("Loading dataset finished")
    return dataset


def build_sentences(dataset, from_, to_):
    print("Building sentences")
```

```
    sentences = [sentence for sentence in dataset["sentence"][from_:to_]]
    print(f"{len(sentences)} sentences created")
    return sentences


def create_embeddings(model, sentences):
    print("Embedding started")
    embeddings = model.encode(sentences)
    print(f"Embeddings shape: {embeddings.shape}")
    print("Embedding finished")
    return embeddings


def create_faiss_index(embeddings):
    print("FAISS index construction started")
    DIMENSIONS = embeddings.shape[1]
    index = faiss.IndexFlatL2(DIMENSIONS)
    index.add(embeddings)
    print(f"Index: {index.ntotal}")
    print("FAISS index construction finished")
    return index


def find_similar_sentences(model, index, query_sentence, k):
    query_embedding = model.encode([query_sentence])
    distances, indices = index.search(query_embedding, k)
    print("-"*40)
    print(f"Query: {query_sentence}")
    print(f"Most {k} similar sentences:")
    for i, idx in enumerate(indices[0]):
        print(f"{i + 1}: {sentences[idx]} (Distance: {distances[0][i]})")


model = initialize_model(MODEL_NAME)
dataset = load_dataset_by_id(DATASET_ID)
sentences = build_sentences(dataset, 0, 1000)
embeddings = create_embeddings(model, sentences)
index = create_faiss_index(embeddings)

find_similar_sentences(model, index, "city", 3)
find_similar_sentences(model, index, "animal", 3)
find_similar_sentences(model, index, "geometry", 3)
find_similar_sentences(model, index, "weather", 3)
find_similar_sentences(model, index, "game", 3)
find_similar_sentences(model, index, "school", 3)
```

PGVector a Sentence transformers

- zápis vektorizovaných vět do tabulky

```
import psycopg2
from sentence_transformers import SentenceTransformer

from pgvector.psycopg2 import register_vector

model = SentenceTransformer("paraphrase-MiniLM-L6-v2")

print(model)

sentences = [
    "The rain in Spain falls mainly on the plain",
    "The tesselated polygon is a special type of polygon",
    "The quick brown fox jumps over the lazy dog",
    "To be or not to be, that is the question",
    "It is a truth universally acknowledged...",
    "How old are you?",
    "The goat ran down the hill"
]

embeddings = model.encode(sentences)
print(f"Embeddings shape: {embeddings.shape}")

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

register_vector(connection)

for vector in embeddings:
    print(type(vector), vector.shape)
    with connection.cursor() as cursor:
        cursor.execute("INSERT INTO v384 (embedding) VALUES (%s)", (vector, ))

connection.commit()
```

Vyhledávání na základě podobnosti

```
import psycopg2
from sentence_transformers import SentenceTransformer

from pgvector.psycopg2 import register_vector

connection = psycopg2.connect(
    host="", port=5432, user="tester", password="123qwe", dbname="test"
)

print(connection)
register_vector(connection)


model = SentenceTransformer("paraphrase-MiniLM-L6-v2")
print(model)


def find_similar_sentences(connection, query_sentence, k):
    print(f"Query: {query_sentence}")
    print(f"Most {k} similar sentences:")

    vector = model.encode(query_sentence)

    with connection.cursor() as cursor:
        query = """
            SELECT id, embedding
              FROM v384
             ORDER BY embedding <-> %s::vector
```

```
            LIMIT %s
        """
        cursor.execute(query, (vector, k))
        records = cursor.fetchall()
        for record in records:
            print(record[0])
    print("-"*40)
```

```
find_similar_sentences(connection, "The quick brown fox jumps over the lazy dog", 3)
find_similar_sentences(connection, "quick brown fox jumps over lazy dog", 3)
find_similar_sentences(connection, "The quick brown fox jumps over the angry dog", 3)
find_similar_sentences(connection, "The quick brown cat jumps over the lazy dog", 3)
find_similar_sentences(connection, "What is your age?", 3)
find_similar_sentences(connection, "Shakespeare", 3)
find_similar_sentences(connection, "animal", 3)
find_similar_sentences(connection, "geometry", 3)
find_similar_sentences(connection, "weather", 3)
```

Realizace celého řetězce

- nyní již známe všechny části řetězce
- jak volat LLM
- jak se vektorizuje text
- jak se zpracovávají dokumenty (chunking) před vektorizací
- jak se provádí sémantické vyhledávání
- zbývá maličkost: vlastní implementace

Konfigurace projektu

```
[project]
name = "docs"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "bs4>=0.0.2",
    "datasets>=4.8.4",
    "faiss-cpu>=1.13.2",
    "langchain>=1.2.13",
    "langchain-classic>=1.0.3",
    "langchain-community>=0.4.1",
    "langchain-openai>=1.1.12",
    "matplotlib>=3.10.8",
    "sentence-transformers>=5.3.0",
]
```

Implementace řetězce s RAGem

- od otázky k odpovědi, včetně RAGu
- provedení série kroků
  1. načtení vstupního textového dokumentu
  2. chunking
  3. konstrukce embedded modelu
  4. ukázka, která část textu se vrátí na položenou otázku
  5. příprava řetězce pro LLM
  6. zavolání LLM se zpracováním RAGu
  7. zobrazení odpovědi

Poznámka: použijeme FAISS, ale stejně tak by bylo možné využít PgVector

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate
from langchain_community.chat_models import ChatOpenAI
from langchain_classic.chains import create_retrieval_chain
from langchain_classic.chains.combine_documents import create_stuff_documents_chain
```

načtení vstupního textového dokumentu

```
DOCUMENT_PATH = "README.TXT"

loader = TextLoader(DOCUMENT_PATH)
text_document=loader.load()
```

chunking

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
documents = text_splitter.split_documents(text_document)
```

konstrukce embedded modelu

```
embeddings = OpenAIEmbeddings()

vectorstore = FAISS.from_documents(documents, embeddings)
```

ukázka, která část textu se vrátí na položenou otázku

```
query = "What is Zonk?"
result = vectorstore.similarity_search(query)

print("Similarity search:")
print(result[0].page_content)
print()
```

příprava řetězce pro LLM

```
prompt = ChatPromptTemplate.from_template(
    "You are an expert assistant. Answer the following question based only on the prov
    "<context>" + \
    "{context}" + \
    "</context>" + \
    "Question: {input}"
)

llm = ChatOpenAI(model_name="gpt-4o-mini")
retriever = vectorstore.as_retriever()
```

```
document_chain = prompt | llm
chain = create_retrieval_chain(retriever, document_chain)
```

zavolání LLM se zpracováním RAGu

```
response = chain.invoke({
    "input": "What is Zonk?",
})
```

zobrazení odpovědi

```
print("And the answer is:")
print(response["answer"])
```

Další možnosti rozšiřování projektu

- MCP (Model Context Protocol)
- Agenti
- Guardrails
- Odlišné technologie pro RAG
- hybridní vyhledávání
- Milvus atd.
- …co dalšího bude objeveno

Příloha: MCP klient

Realizace klienta.

```
from time import time

from mcp import ClientSession
from mcp.client.sse import sse_client


async def run(i):


    async with sse_client(url="http://localhost:8000/sse") as (read, write):
        async with ClientSession(read, write) as session:
            t1 = time()
            print(f"Client #{i} initialization")
            print()

            await session.initialize()
```

přečtení zdroje bez selektoru

```
            data = await session.read_resource("pozdrav://")
            print("Data returned:", data)
            print("Type:", type(data))
            text = data.contents[0].text
            print("Text:", text)
            print()
```

přečtení zdroje se selektorem

```
            data = await session.read_resource("pozdrav://Pavel")
            print("Data returned:", data)
            print("Type:", type(data))

            text = data.contents[0].text
            print("Text:", text)
            print()
```

zavolání nástroje

```
            data = await session.call_tool("factorial", arguments={"n": 10})
            factorial = data.content[0].text
            print("10!=", factorial)
            print()

            t2 = time()
            difftime = t2 - t1
            print(f"Client has finished in {difftime} seconds")
            print()


async def main():
    clients = [run(i) for i in range(10)]
    await asyncio.gather(*clients)


if __name__ == "__main__":
    import asyncio

    asyncio.run(main())
```

MCP server

```
from time import sleep

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Test")

SLEEP_AMOUNT=0.5
```

Výpočet faktoriálu ve smyčce.

```
@mcp.tool()
def factorial(n: int) -> int:


    print("Factorial computation started")
    f = 1
    for x in range(1, n + 1):
        f *= x
    sleep(SLEEP_AMOUNT)
    print("Factorial computation finished")
    return f
```

Odpověď s pozdravem.

```
@mcp.resource("pozdrav://")
def pozdrav1() -> str:


    print("Resource preparation started")
```

Odpověď s osobním pozdravem.

```
    sleep(SLEEP_AMOUNT)
    print("Resource preparation finished")
    return "Hello, dear client!"


@mcp.resource("pozdrav://{name}")
def pozdrav2(name: str) -> str:


    print("Resource preparation started")
    sleep(SLEEP_AMOUNT)
    print("Resource preparation finished")
    return f"Hello, {name}!"


if __name__ == "__main__":
    mcp.run(transport="sse")
```

Děkuji za pozornost